

数据可视化

北京理工大学

数据可视化其实就是通过图形来比较直观地展示数据，好处在于直观、醒目、简单。在本章中，我们主要学习可视化模块 `matplotlib`。在 `matplotlib` 模块中，应用最广的绘图工具包是 `matplotlib.pyplot`。这是绘制平面 (二维) 图形的常用模块。这个模块的画图风格非常接近于 `MATLAB`，命令的使用方式与 `MATLAB` 中的画图命令也是差不多的。因此在为该模块取名时，就使用到了 `MATLAB` 的前三个字母。这个模块名字中间的 `plot` 表示绘图功能，而末尾的 `lib` 则表示这是一个库，集成了许多功能。

本章将以 matplotlib.pyplot 为基础，主要讲述 6 种基础统计图形的绘制方法：散点图、折线图、柱状图、饼图、箱线图和概率图。另外，我们也会简单地提一下直方图。先导入 matplotlib.pyplot 模块，并缩写为 plt。另外，导入 numpy 和 pandas 模块，分别缩写为 np 和 pd。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

大体说来，绘制图形有三个步骤。第一步，构建画图区域。可以在某个区域中画一副或多幅图形，常用的三个函数如下。

Table: 构建画图区域函数

函数名称	函数作用
plt.figure	在指定区域画一副图形，可以指定区域大小、像素。
plt.subplot	在指定区域画多副子图，可以指定子图行列数和编号。
plt.add_subplot	在指定区域画多副子图，可以指定子图行列数和编号。

第二步一般认为是绘制图形的主要工作，包括根据数据绘制指定图形、添加标题和坐标轴名称等等。我们可以先绘制图形，也可以先添加各类标签。但是添加图例一定要在绘制图形之后。这一步经常使用的函数如下。

Table: 添加标签函数

函数名称	函数作用
plt.title	在当前图形中添加标题。
plt.xlabel	在当前图形中添加横轴名称。
plt.ylabel	在当前图形中添加纵轴名称。
plt.xlim	当前图形横轴的范围。
plt.ylim	当前图形纵轴的范围。
plt.xticks	横轴刻度的数目与数值。
plt.yticks	纵轴刻度的数目与数值。
plt.legend	当前图形的图例，可以指定图例的大小、位置、标签。

第三步则是保存和显示图形了。常用函数有如下两个。

Table: 保存和显示函数

函数名称	函数作用
<code>plt.savefig</code>	保存绘制的图形。
<code>plt.show</code>	在本机显示图形。

最简单的绘图可以省略第一部分，直接在默认的区域绘制图形。绘制图形涉及到许多参数。在多数情况下，这些参数都会有默认取值。如果使用手动设置的话，往往可以绘制更加个性化的图形。

本节主要介绍在 Python 中绘制散点图和折线图的函数常用参数。

散点图利用坐标点的分布形态反映特征间的统计关系。绘制散点图的函数为 `scatter`，其函数常用参数及说明如下表所示。

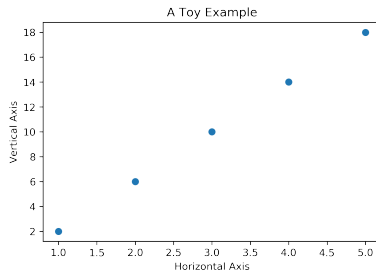
Table: 散点图函数常用参数

参数名称	说明
x,y	数组 (array)。表示横轴和纵轴对应的数据。
s	数值或一维数组 (array)。指定大小。
c	数值或一维数组 (array)。指定颜色。
marker	特定字符串 string。指定点的类型。
plt.ylim	指定当前图形纵轴的范围。
alpha	接收 0~1 的小数。表示点的透明度。

下面我们使用 `scatter` 函数来画一个简单的散点图。调用 `scatter` 函数时，第一个参数给出横轴的取值，第二个参数给出纵轴的取值。两个参数的长度必须一样，否则会报错的。同时，我们给出横轴、纵轴的标签及标题。顺便提一下，大家暂时不要尝试使用中文来表示纵、横轴的标签或标题。尽管程序不会报错，但是中文有时候不能正确显示。以后，我们会有相关的案例告诉大家如何使用中文来表示纵、横轴的标签或标题。

```
plt.scatter ([1,2,3,4,5], np.arange(2,20,4))  
plt.xlabel('Horizontal Axis') # 添加x轴标签  
plt.ylabel('Vertical Axis') # 添加y轴标签  
plt.title ('A Toy Example') # 添加标题  
plt.show()
```

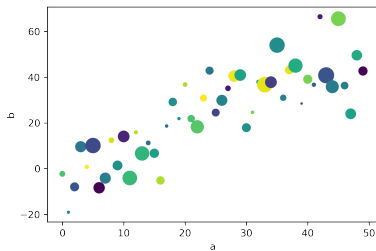

Figure: 散点图示例



下面我们来看看 scatter 这个函数的输出控制。我们可以通过字符串对应的关键词来提取一些数据。我们可以看看下面的例子。

```
data = {'a': np.arange(50),  
        'c': np.random.randint(0,50,50),  
        'd': np.random.randn(50)}  
data['b'] = data['a'] + 10*np.random.randn(50)  
data['d'] = np.abs(data['d']) * 100  
plt.scatter('a','b', c = 'c', s = 'd', data = data)  
plt.xlabel('a')  
plt.ylabel('b')  
plt.show()
```

Figure: 散点图示例 2



折线图包含了散点图作为特殊情形。

折线图是一种将数据点成对展示并连接起来的图形。我们可以把折线图看作是将散点图按照横轴坐标顺序连接起来的图形。折线图主要是查看纵轴变量随着横轴变量变化的趋势，绘制折线图的函数为 `plot`。调用 `plot` 函数时，可以画折线图或散点图。用 `plot` 函数来画折线图，第一个参数给出横轴的取值，第二个参数给出纵轴的取值。两个参数的长度必须一样。

Table: 折线图函数常用参数

参数名称	说明
x,y	数组 (array)。表示横轴和纵轴对应的数据。
color	特定字符串 string。指定线条颜色。
linestyle	特定字符串 string。指定线条类型。
marker	特定字符串 string。表示点的类型。
alpha	接收 0~1 之间的小数。表示点的透明度。

主要输入参数如上表所示。其中，指定 color 参数时，有 8 种常用颜色可以选择：“b”是蓝色，“g”是绿色，“r”是红色，“c”是青色，“m”是品红色，“y”是黄色，“k”是黑色，“w”是白色。

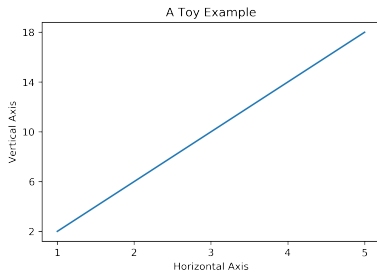
下面我们来看一个简单的例子，使用 plot 函数来画一个简单的折线图。

```
# 绘制折线
plt.plot ([1,2,3,4,5], np.arange(2,20,4))

# 添加坐标轴以及标题
plt.xticks ([1,2,3,4,5])
plt.yticks (np.arange(2,20,4))
plt.xlabel('Horizontal Axis')
plt.ylabel('Vertical Axis')
plt.title ('A Toy Example')

# 显示图像
plt.show()
```

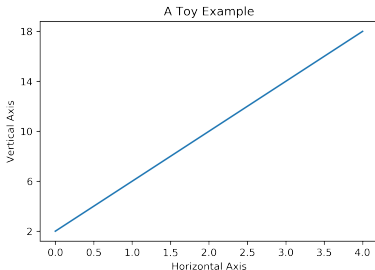
Figure: 折线图示例



在 plot 函数中，如果只给出一个参数的话，默认这个参数给出了纵轴的取值范围，横轴会自动产生一个从 0 开始的列表与之对应。大家可以对比一下下面的图形和上面的图形的横轴取值的差别。

```
plt.plot(np.arange(2,20,4))  
plt.xlabel('Horizontal Axis')  
plt.ylabel('Vertical Axis')  
plt.title('A Toy Example')  
plt.show()
```

Figure: 折线图示例 2

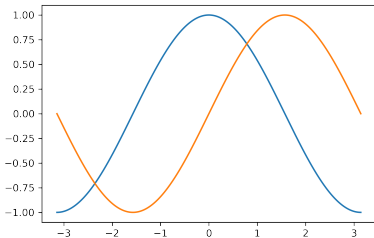


可以看到，尽管我们没有指定横轴的范围，但是画出的图形给出了 0 到 4 这个数值范围。一般情况下，如果我们只是指定了纵轴的取值范围，横轴会自动指定一系列从 0 开始的数组与纵轴相对应。画图函数 `plot` 非常灵活，我们可以按照自己的意图指定横轴以及纵轴的范围。我们可以不指定横轴的取值，但如果我们一旦指定横轴的取值，横轴的列表长度就必须和纵轴的列表长度完全一样长。

下面我们来看一个 $\sin$ 函数以及一个 $\cos$ 函数。我们先产生一个 numpy 数组，包含了从 $-\pi$ 到 π 等间隔的 256 个值。 $\cos$ 和 $\sin$ 则分别是这 256 个值对应的余弦和正弦函数值。

```
X = np.linspace(-np.pi, np.pi, 256, endpoint=True)
C,S = np.cos(X), np.sin(X)
plt.plot(X, C) # 绘制轴为的曲线yC
plt.plot(X, S) # 绘制轴为的曲线yS
plt.show()
```

Figure: 正弦和余弦图



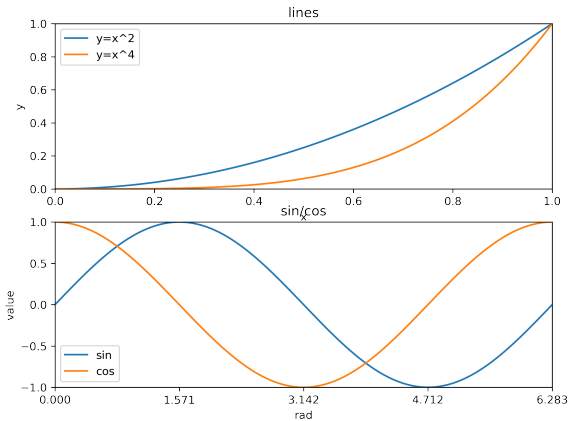
下面来看几个绘制子图的例子。

```
rad = np.arange(0, np.pi*2, 0.01)
p1 = plt.figure(figsize=(8, 6), dpi=80) # 设置绘图大小与分辨率
# 绘制第一张子图
ax1 = p1.add_subplot(2, 1, 1)
plt.title('lines')
plt.xlabel('x')
plt.ylabel('y')
plt.xlim(0,1)
plt.ylim(0,1)
plt.xticks([0, 0.2, 0.4, 0.6, 0.8, 1])
plt.yticks([0, 0.2, 0.4, 0.6, 0.8, 1])
plt.plot(rad, rad**2)
plt.plot(rad, rad**4)
plt.legend(['y=x^2', 'y=x^4'])
```

绘制子图

```
# 绘制第二张子图
ax2 = p1.add_subplot(2,1,2)
plt.title('sin/cos')
plt.xlabel('rad')
plt.ylabel('value')
plt.xlim(0, np.pi*2)
plt.ylim(-1, 1)
plt.xticks([0, np.pi/2, np.pi, np.pi*1.5, np.pi*2])
plt.yticks([-1, -0.5, 0, 0.5, 1])
plt.plot(rad, np.sin(rad))
plt.plot(rad, np.cos(rad))
plt.legend(['sin', 'cos'])
plt.show()
```

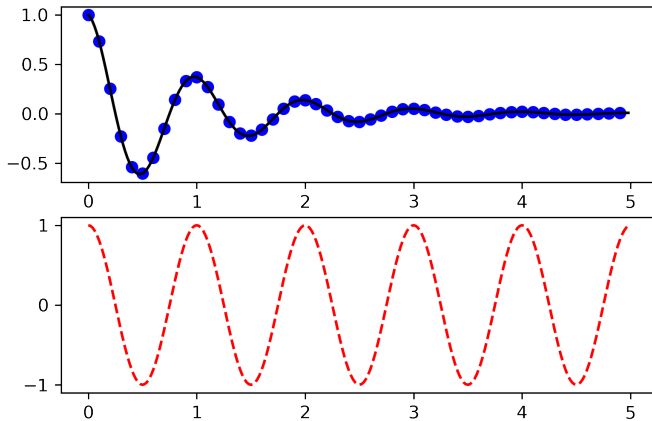
Figure: 子图示例



我们给出第二种画子图的例子。

```
# 生成数据
def f(t):
    return np.exp(-t)*np.cos(2*np.pi*t)
t1 = np.arange(0,5, 0.10)
t2 = np.arange(0,5, 0.02)
# 绘制子图
plt.figure(1)
plt.subplot(211)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')
plt.subplot(212)
plt.plot(t2, np.cos(2*np.pi*t2), 'r--')
plt.show()
```

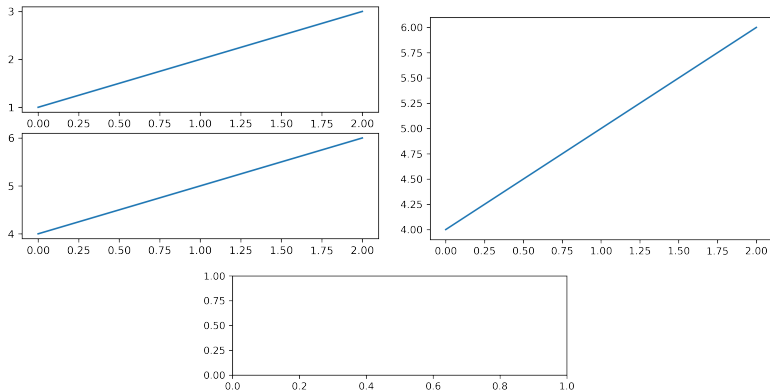
Figure: 子图示例 2



在上面的脚本程序中，figure(1) 这个指令其实是没有必要的，因为默认地我们会产生 figure(1)。我们也可以用 figure() 来产生更多的图形。绘制子图的函数 subplot() 指定行数、列数以及子图的序号。类似的，我们默认产生 subplot(111)。很显然，子图的序号应该小于等于行数与列数的乘积。只要行数乘以列数小于 10，subplot(211) 就会等价于 subplot(2,1,1)。我们来看看下面这段程序脚本产生的图形效果的差异。

```
plt.figure(1)
plt.subplot(211)
plt.plot([1, 2, 3])
plt.subplot(212)
plt.plot([4, 5, 6])
plt.figure(2)
plt.plot([4, 5, 6])
plt.figure(3)
plt.subplot(211)
plt.show()
```

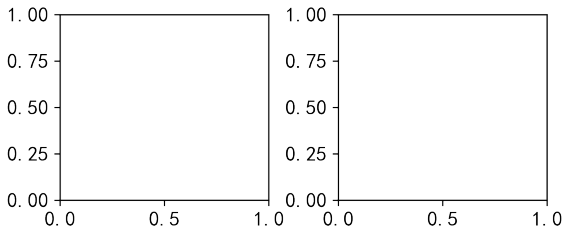

Figure: 子图示例 3



如果希望产生一个坐标系，可以使用命令 `axes([left,bottom,width,height])`，所有的取值都在 0 到 1 之间。这个函数与 `subplot()` 是差不多的。但利用 `axes()` 产生的这些子图形的位置和大小可以自主设置，不一定会像 `subplot()` 一样会产生一个长方形的子图形。另外，我们要特别小心区分如下两个函数：`plt.axis([xmin,xmax,ymin,ymax])` 与 `plt.axes([left,bottom,width,height])`。我们可以对比一下下面这个脚本程序画图效果的差异，并通过这些差异来领会命令行的意思。

```
plt.axes([0.2, 0.4, 0.3, 0.4])  
plt.axes([0.6, 0.4, 0.3, 0.4])  
plt.show()
```

Figure: 子图示例 4



可以利用 `clf()` 来清除当前的图形，也可以利用 `cla()` 来清除当前的坐标系。如果产生了许多图形，除非使用 `close()` 来完全关闭了这个图形，否则的话这个图形所占用的内存不会被完全释放的。

在上面的例子中，我们主要采用 matplotlib 的默认配置。这些默认配置在大多数情况下已经做得足够好，一般情况下我们无须更改这些默认配置。但是，这些默认配置大多数可以允许更改，包括图形大小和分辨率 (dpi)、线条的宽度、颜色、风格、坐标轴、坐标轴以及网格的属性、文字与字体属性等。比如说，对于线条的配置可以做如下更改。

Table: 线条配置参数

参数名称	解释	取值
lines.linewidth	线条宽度	取 0~10 之间的数值，默认为 1.5。
lines.linestyle	线条样式	可取 “-”, “-”, “-.” 和 “:” 四种，默认 “-”。
lines.marker	线条上点的形状	取 “o”, “D” 和 “H” 等 20 来种，默认 None。
lines.markersize	点的大小	取 0~10 之间的数值，默认为 1。

其中，线条样式“-”代表实线，“-”代表长虚线，“-.”代表点线，“:”代表短虚线。关于线条上的点，有如下形状。

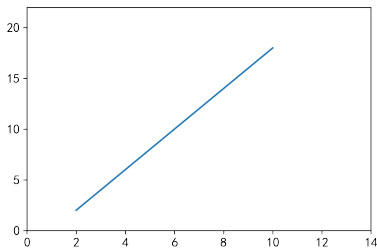
Table: 点形状参数

取值	意义	取值	意义	取值	意义	取值	意义
o	圆圈	.	点	D	菱形	s	正方形
h	六边形 1	*	星号	H	六边形 2	d	小菱形
-	水平线	v	角朝下的三角形	8	八边形	<	角朝左的三角形
p	五边形	>	角朝右的三角形	,	像素	^	角朝上的三角形
+	加号	\	竖线	None	无	x	X

下面来看一些例子。首先我们可以看看如何指定线条的类型与颜色。使用的方式与 Matlab 的使用方式是一样的。默认的是命令是蓝色实线'b-'。

```
plt.plot(np.arange(2, 12, 2), np.arange(2, 20, 4))  
plt.axis([0, 14, 0, 22])  
plt.show()
```

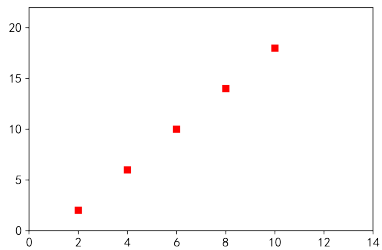
Figure: 蓝色实线示例



在上面的图形中，我们使用了 `axis` 来控制横轴以及纵轴的取值范围，`axis` 函数分别要求输入横轴极小值、横轴极大值、纵轴极小值、纵轴极大值。注意到，`plot` 函数默认输入是一个蓝色的线条。但是，我们完全可以更改 `plot` 函数的输出，比如，我们可以要求用红色的方块来显示这些数据。

```
plt.plot(np.arange(2, 12, 2), np.arange(2, 20, 4), 'rs')  
plt.axis([0, 14, 0, 22])  
plt.show()
```

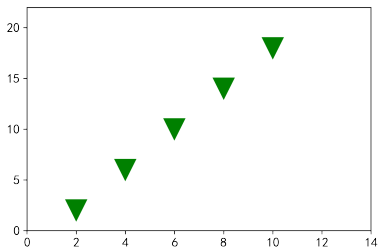
Figure: 红色方块点示例



可以尝试不同的点的形状，我们通过 `markersize` 来控制点的大小。

```
plt.plot(np.arange(2, 12, 2), np.arange(2, 20, 4), 'gv', markersize = 20)  
plt.axis([0, 14, 0, 22])  
plt.show()
```

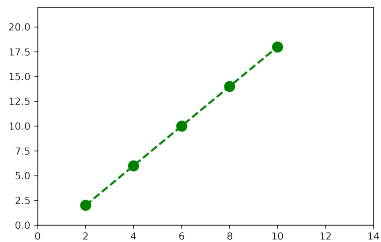
Figure: 修改点的形状与大小示例



下面我们可以要求用绿色（黄色、红色、蓝色等）的虚线来表达。我们可以通过 `linewidth` 来控制线条的宽度，通过 `linestyle` 来控制线条的形状。

```
plt.plot(np.arange(2, 12, 2), np.arange(2, 20, 4), 'g.--', linewidth = 2)
plt.axis([0, 14, 0, 22])
plt.show()
```

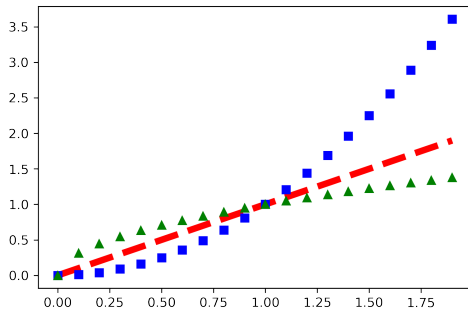
Figure: 线条形状示例



我们可以在一个图形中绘制多个不同的线条。关于这一点，在前面的例子中我们已经展示了。我们可以再看一个例子。

```
t = np.arange(0, 2, 0.1)
plt.plot(t, t, 'r--', linewidth = 5)
plt.plot(t, t**2.0, 'bs')
plt.plot(t, t**0.5, 'g^')
plt.show()
```

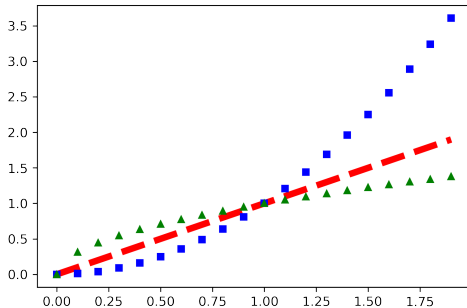
Figure: 多线条示例



对于上面的程序脚本，可以使用一行命令实现不同的画图风格。

```
t = np.arange(0, 2, 0.1)
plt.plot(t, t, 'r--',
         t, t**2.0, 'bs',
         t, t**0.5, 'g^', linewidth = 5, markersize = 5)
plt.show()
```

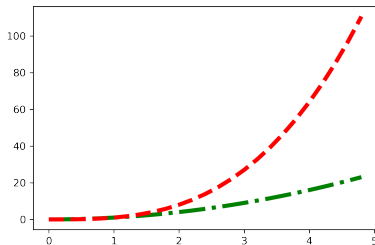
Figure: 多线条示例 2



我们可以通过一些关键词来控制线条的宽度，线条的类型等。

```
t = np.arange(0, 5, 0.2)
plt.plot(t, t**2, color = 'g', linestyle='-.', linewidth = 4.0)
plt.plot(t, t**3, color = 'r', linestyle='--', linewidth = 4.0)
plt.show()
```

Figure: 多线条示例 3



我们也可以通过 `setp()` 这个命令来获取这些线条的性质。由于命令行的输出结果过长，这里就不再列出输出结果了。

```
lines = plt.plot([1, 2, 3])  
plt.setp(lines)
```


开普勒定律是典型的基于数据的方法来开展科学研究的案例。开普勒定律是德国天文学家开普勒发现的关于行星运动的规律特征。1609 年,《新天文学》发表了开普勒关于行星运动的前两条定律;1618 年,开普勒提出了第三条定律。这三大定律又分别称为椭圆定律、面积定律和调和定律。

- ① 椭圆定律: 所有行星绕太阳的轨道都是椭圆形的, 太阳位于椭圆形的其中一个焦点上。
- ② 面积定律: 行星和太阳的连线在相等的时间间隔内扫过相等的面积。
- ③ 调和定律: 所有行星绕太阳一周的恒星时间的平方与它们轨道长半轴的立方成比例。

开普勒的三大定律是根据他的前任，一位叫第谷的天文学家留给他的观察数据总结出来的。丹麦著名的天文学家第谷·布拉赫花费了超过 20 多年的时间，观察与收集了大量非常精确的天文资料。大约于 1605 年，根据第谷的行星位置资料，沿用哥白尼的匀速圆周运动理论，通过 4 年的计算，开普勒发现第谷观测到的数据与计算有 8' 的误差，开普勒坚信第谷观测到的数据是正确的，从而他对“完美”匀速圆周运动发起质疑。经过近 6 年的大量计算，开普勒提出了第一定律和第二定律，又经过 10 年的大量计算，得出了第三定律。

开普勒定律对亚里士多德派与托勒密派构成了极大的挑战。开普勒主张地球是不断移动的；行星轨道不是圆周形而是椭圆形的；行星公转的速度并不恒定。这些论点动摇了当时的天文学与物理学的基础。经过了几乎一个世纪披星戴月、废寝忘食的研究，物理学家终于能够用物理理论解释其中的道理。牛顿利用他的第二定律和万有引力定律，在数学上严格地证明开普勒定律，也让人们了解其中的物理意义。

下面我们给出的观测数据是行星绕太阳一周所需要的时间（以年为单位）和行星离太阳的平均距离（以地球与太阳的平均距离为单位）。

```
a = ['水星','金星','地球','火星','木星','土星','天王星','海王星']
b = [0.241,0.615,1.00,1.88,11.8,29.5,84.0,165]
c = [0.39,0.72,1.00,1.52,5.20,9.54,19.18,30.06]
table = pd.DataFrame({'行星': a, '周期（年）': b, '平均距离': c},
columns = ['行星','周期（年）','平均距离'])
table['周期平方/距离立方'] = table['周期（年）']**2/table['平均距离']**3
table
```

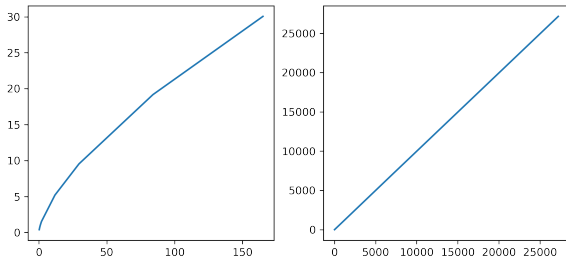
Table: 行星观测数据

	行星	周期 (年)	平均距离	周期平方/距离立方
0	水星	0.241	0.39	0.97913
1	金星	0.615	0.72	1.013334
2	地球	1.000	1.00	1.000000
3	火星	1.88	1.52	1.006433
4	木星	11.8	5.2	0.990271
5	土星	29.5	9.54	1.002303
6	天王星	84	19.18	1.000029
7	海王星	165	30.06	1.002307

从这组数据可以看出，行星绕太阳运行的周期的平方和行星离太阳的平均距离的立方成正比，这就是开普勒第三定律。

```
plt.figure(1, figsize = (9,4))  
plt.subplot(121)  
plt.plot(table['周期 (年)'], table['平均距离'])  
plt.subplot(122)  
plt.plot(table['周期 (年)']**2, table['平均距离']**3)  
plt.show()
```

Figure: 行星周期与平均距离关系图



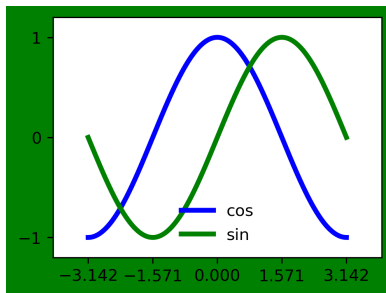
我们再来看一个稍微复杂一点的例子。一般的图形的大小为 4×3 点，但是，可以通过 `figsize=(8,6)` 更改图形的大小为 8×6 点。如果我们希望生成一个比较长的图形，可以设置 `figsize=(12,6)` 来增加宽度。同时，可以通过 `dpi = 80` 来控制图形的分辨率为 80。

```
plt.figure(figsize=(8,6), dpi=80, facecolor='g')
plt.subplot(2,2,1)
X = np.linspace(-np.pi, np.pi, 256, endpoint=True)
C,S = np.cos(X), np.sin(X)
plt.plot(X, C, color="blue", linewidth=3.0, linestyle="--")
plt.plot(X, S, color="green", linewidth=3.0, linestyle="--")
plt.xlim(-4.0, 4.0)
plt.ylim(-1.2,1.2)

plt.xticks(np.linspace(-np.pi, np.pi, 5, endpoint=True))
plt.yticks(np.linspace(-1, 1, 3, endpoint=True))

plt.savefig("SinCos.png", dpi=72)
# plt.legend(['cos', 'sin'], loc='best')
plt.legend(['cos', 'sin'], loc='lower center', frameon=False)
plt.show()
```


Figure: 正弦和余弦图

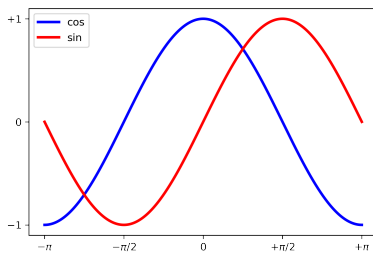


添加图例的时候，我们可以在 plot 中增加一个参数 legend。如果忘记了 legend 函数中 loc 的可选参数，可以随便给一个值。这样，程序运行的时候可能会报错，但同时会提示可选的取值。一般我们可以取值 loc = best。

另外，上面的图形中，横轴 xticks 给出的标签不是非常理想，而且不够精确。我们可以使用 LaTeX 命令来设置标签。

```
plt.plot(X, C, color="blue", linewidth=2.5, linestyle="-", label="cos")
plt.plot(X, S, color="red", linewidth=2.5, linestyle="-", label="sin")
plt.legend(loc='upper left')
plt.xticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi],
           [r'$-\pi$', r'$-\pi/2$', r'$0$', r'$+\pi/2$', r'$+\pi$'])
plt.yticks([-1, 0, +1],
           [r'$-1$', r'$0$', r'$+1$'])
plt.show()
```

Figure: 正弦和余弦图 2

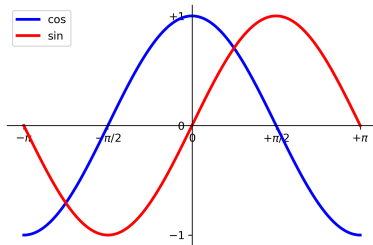


在所有的图形中，坐标轴线和上面的记号记录了数据区域的范围，连在一起就形成了 spines。它们可以放在任意位置，不过至今为止，我们都把它放在图的四边。实际上每幅图有四条 spines（上、下、左、右），为了将 spines 放在图的中间，我们必须将其中的两条（上和右）设置为无色，然后调整剩下的两条到合适的位置——数据空间的 0 点。

```
ax = plt.gca()
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data', 0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data', 0))

plt.show()
```

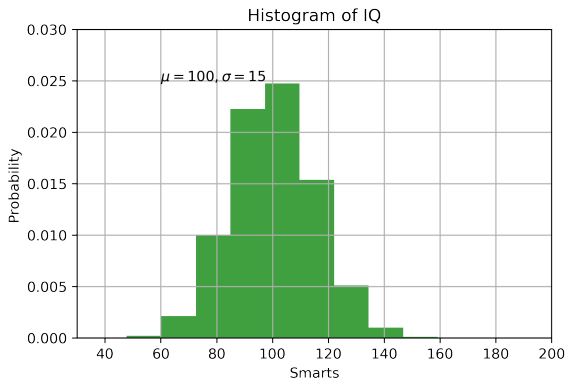
Figure: 正弦和余弦图 3



利用 `text()` 这个命令可以在任意一个位置产生一些文本, `xlabel()`, `ylabel()` 以及 `title()` 可以分别在不同的位置产生文本。

```
mu, sigma = 100, 15
x = mu + sigma * np.random.randn(10000)
n,bins,patches = plt.hist(x, 50, density = 1, facecolor = 'g', alpha = 0.75)
plt.xlabel('Smarts')
plt.ylabel('Probability')
plt.title('Histogram of IQ')
plt.text(60, 0.025, r'$\mu = 100, \sigma = 15$')
plt.axis([40, 160, 0, 0.03])
plt.grid(True)
plt.show()
```

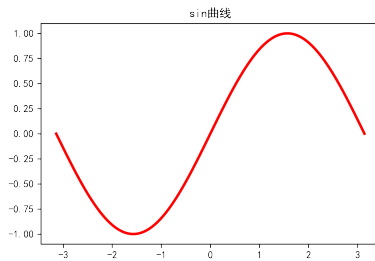
Figure: IQ 直方图



由于默认的 pyplot 字体并不支持中文字符的显示，因此需要通过设置 `font.sans-serif` 参数来改变绘图时的字体，使得图形可以正常显示中文。同时，由于更改字体后，会导致坐标轴中的部分字符无法显示，因此需要同时更改 `axes.unicode_minus` 参数。下面看一个例子。

```
plt.rcParams['font.sans-serif']='SimHei'  
plt.rcParams['axes.unicode_minus']=False  
plt.plot(X, S, color="red", linewidth=2.5, linestyle="--", label="sin")  
plt.title('sin曲线')  
plt.show()
```

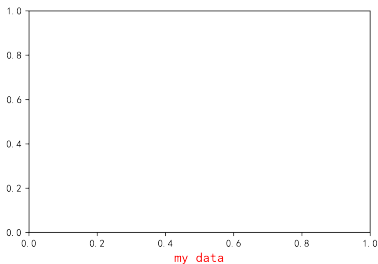

Figure: 中文显示示例



我们也可以使用 `setp()` 来做一些个性化设置。另外，我们可以根据关键词来做一些个性化设置。

```
t = plt.xlabel('my data', fontsize = 14, color = 'red')  
plt.show()
```

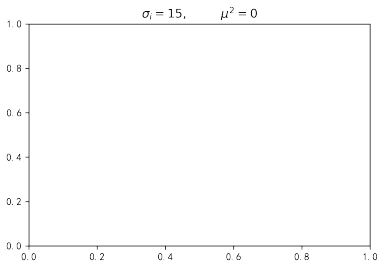
Figure: 个性化设置示例



在 `text()` 中可以接收 Tex 格式的表达式，只需要把这些表达式放在 `$` 符号里即可。另外，字符串前面的 `r` 也是非常重要的，`r` 表明后面的 `\` 不是表达空格的意思，而是用在 Tex 表达式中。我们可以看看下面的例子。

```
plt.title(r'$\sigma_i = 15, \ \mu^2 = 0$')  
plt.show()
```

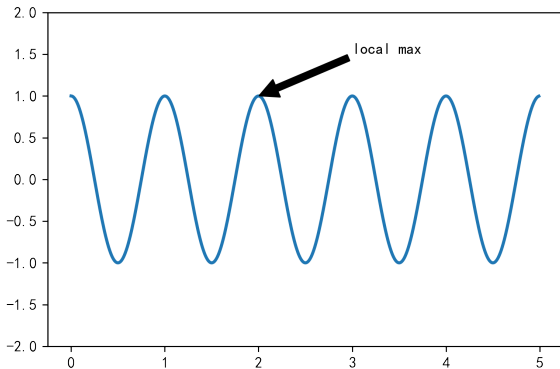
Figure: `text()` 用法示例



使用 `annotate()` 命令可以在坐标系的任何一个位置放置文本。文本比较常用的用途是对图形加一些说明。使用 `annotate()` 这个函数有两个地方需要注意：通过 `xy` 来指定需要进行注释的图形位置，用 `xytext` 来指定注释文本的位置，这两个都是使用 `(x,y)` 元组数据。请看下面的例子。

```
ax = plt.subplot(111)
t = np.arange(0.0,5.0,0.01)
s = np.cos(2*np.pi*t)
line = plt.plot(t,s,lw = 2)
plt.annotate('local max', xy = (2,1), xytext = (3,1.5),
            arrowprops = dict(facecolor = 'black',shrink = 0.05))
plt.ylim(-2,2)
plt.show()
```

Figure: 文本放置示例



在这个很简单的例子中，`xy()` 以及 `xytext()` 都是使用坐标系的绝对位置，不是相对位置。

另外，`annotate` 函数中的 `shrink` 参数表示箭头总长度“缩水”的比例。

我们再来看一个比较复杂的例子。我们希望在某个给定的位置加上一个注释。首先，我们在对应的函数图像位置上画一个点；然后，向横轴引一条垂线，以虚线标记；最后，写上标签。

```
X = np.linspace(-np.pi, np.pi, 256, endpoint=True)
C, S = np.cos(X), np.sin(X)
```

```
plt.plot(X, C, color="blue", linewidth=2.5, linestyle="-",
         label="cos")
plt.plot(X, S, color="red", linewidth=2.5, linestyle="-",
         label="sin")
plt.legend(loc='upper left')
plt.xticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi],
          [r'$-\pi$', r'$-\pi/2$', r'$0$', r'$+\pi/2$',
           r'$+\pi$'])
plt.yticks([-1, 0, +1],
          [r'$-1$', r'$0$', r'$+1$'])
ax = plt.gca()
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data',0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data',0))
# 上面的程序保持不动，下面的程序控制spines。
```

```
t = 2*np.pi/3
plt.plot([t,t],[0,np.cos(t)], color='blue', linewidth=2.5,
         linestyle="--")
plt.scatter([t],[np.cos(t)], 50, color='blue')

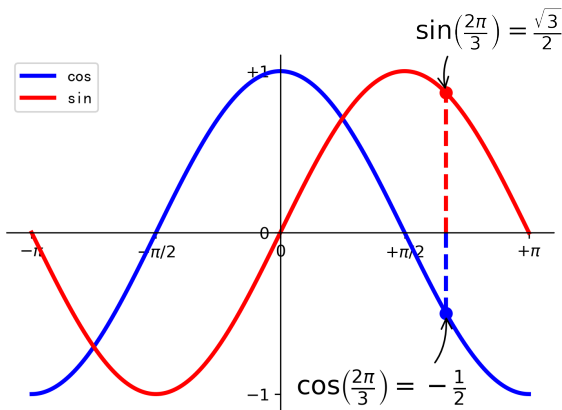
plt.annotate(r'$\sin\left(\frac{2\pi}{3}\right)=\frac{\sqrt{3}}{2}$',
            xy=(t, np.sin(t)), xytext=(1.7, 1.2), fontsize=16,
            arrowprops=dict(arrowstyle="->",
                            connectionstyle="arc3,rad=0.2"))

plt.plot([t,t],[0,np.sin(t)], color='red', linewidth=2.5,
         linestyle="--")
plt.scatter([t],[np.sin(t)], 50, color='red')

plt.annotate(r'$\cos\left(\frac{2\pi}{3}\right)=-\frac{1}{2}$',
            xy=(t, np.cos(t)), xytext=(0.2, -1), fontsize=16,
            arrowprops=dict(arrowstyle="->",
                            connectionstyle="arc3,rad=.2"))

plt.show()
```

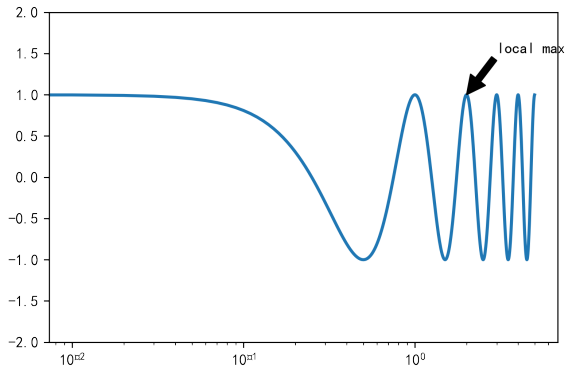

Figure: 非线性坐标系示例



matplotlib.pyplot 不仅可以使⤵线性坐标系，还支持对数以及 logit 刻度。当数据的跨度非常大的时候，这种非线性坐标系就非常有用⤵。使用这种非线性坐标系是很容易的。我们可以简单地如下使用这些非线性刻度。

```
ax = plt.subplot(111)
t = np.arange(0.0,5.0,0.01)
s = np.cos(2*np.pi*t)
line = plt.plot(t,s,lw = 2)
plt.annotate('local max', xy = (2,1), xytext = (3,1.5),
             arrowprops = dict(facecolor = 'black', shrink = 0.05))
plt.ylim(-2,2)
plt.xscale('log')
plt.show()
```

Figure: 非线性坐标系示例 2



柱状图是由一系列高度不等的条纹或线段表示数据的分布状态，横轴一般表示类型，纵轴表示数量或占比。在 pyplot 中，绘制柱状图的函数为 bar 和 barh，其常用的参数及其说明如下表所示。

Table: 柱状图函数常用参数

参数名称	说明
left	数组 (array)。表示横轴数据。
height	数组 (array)。表示横轴所代表数据的数量。
width	0 到 1 之间的浮点数。指定柱状图宽度。
color	特定字符串 (string) 或者颜色字符串的数组 (array)。表示柱状图颜色。

分类变量的例子有许多，比如个体是冰淇淋，分类变量可以是冰淇淋的口味；有些时候我们还使用数值 0, 1, 2 等来表达这些分类变量。

柱状图分为水平柱状图和垂直柱状图。我们先来看看水平柱状图。

```
import pandas as pd
icecream = pd.DataFrame({'Flavor': ['Chocolate', 'Strawberry', 'Vanilla'],
                        'Number of Cartons': [16, 5, 9]})
icecream
```

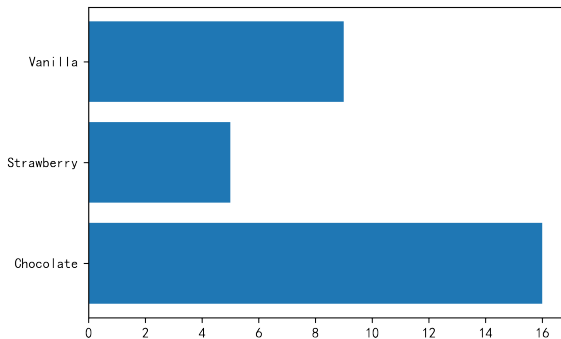
Table: 冰淇淋数据

	Flavor	Number of Cartons
0	Chocolate	16
1	Strawberry	5
2	Vanilla	9

柱状图

```
plt.barh(icecream['Flavor'], icecream['Number of Cartons'])  
plt.show()
```

Figure: 冰淇淋数据水平柱状图



柱状图与之前我们介绍的散点图和折线图除了视觉上的差别之外，还有一个不太一样的地方在于，柱状图的特点是一个数轴是分类变量，另一个数轴是频数；散点图和折线图的两个轴都是数值型数据。柱状图的宽度以及各个柱子之间的间隙都可以自己定义，只要保证各个柱状图的宽度一样，而且各个柱子之间的间隙也是一样的就好了。当只有三个类别的时候，用柱状图和用表格展示数据没有什么差别。但是，当有特别多类别的时候，用柱状图和用表格展示数据的区别就非常大了。

电影公司数据

```
studios = np.array(['Warner Bros','Buena Vista (Disney)','Fox', 'Paramount',  
'Universal','Disney','Columbia','MGM','UA','Sony','New Line',  
'Paramount/Dreamworks','RKO','Lionsgate','Dreamworks','Tris','Sum',  
'Waner Bros. (New Line)', 'Selz','Orion','NM','MPC','IFC','AVCO'])  
counts = np.array ([29,29,26,25,22,11,10,7,6,6,5,4,3,3,3,2,2,1,1,1,1,1,1])  
  
movies_and_studios = pd.DataFrame({'Studio': studios, 'Count': counts})  
movies_and_studios.sort_index(axis = 1, ascending = False, inplace = True)  
movies_and_studios.sort_values(by = 'Count', inplace = True, ascending = False)  
movies_and_studios.head(3)
```


导入数据以后，我们查看一下前 3 行。

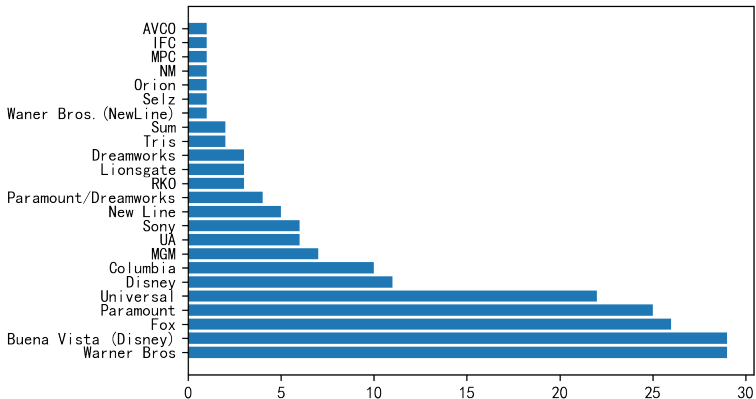
Table: 电影公司数据

	Studio	Count
0	Warner Bros	29
1	Buena Vista (Disney)	29
2	Fox	26

我们按照原始数据的顺序，来画一下水平柱状图。

```
barplot = plt.barh(movies_and_studios['Studio'], movies_and_studios['Count'])  
plt.show()
```

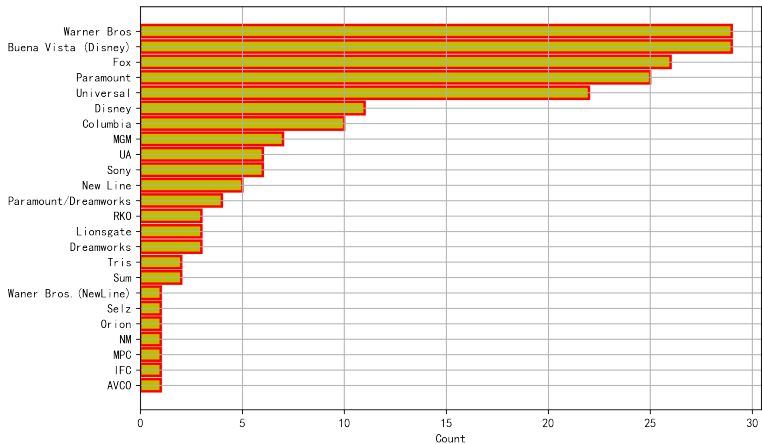
Figure: 电影公司数据水平柱状图



如果我们希望按照从小到大的顺序来展示数据的话，我们可以这么做。

```
plt.figure(figsize=(9, 6))
x_pos = np.arange(len(movies_and_studios.index), 0, -1)
barplot = plt.barh(x_pos, movies_and_studios['Count'],
height = 0.8, left = 0, color= 'y', edgecolor = 'r', linewidth = 2.0)
plt.yticks(x_pos, movies_and_studios['Studio'])
plt.xlabel('Count')
plt.grid(True)
plt.show()
```

Figure: 电影公司数据水平柱状图 2



接下来，我们来看垂直柱状图。绘制垂直柱形图的函数形式为 `bar(left, height, width, bottom, color, align, yerr)`。其中，水平柱形图四个顶角的位置分别为 `left`, `left+width`, `bottom` 以及 `bottom + height`。其中，`left` 为柱形图左边沿在 `x` 轴的位置序列，一般采用 `arange` 函数产生一个序列；`height` 为柱形图沿着 `y` 轴的高度数值序列，也就是柱形图的高度，一般就是我们需要展示的数据；

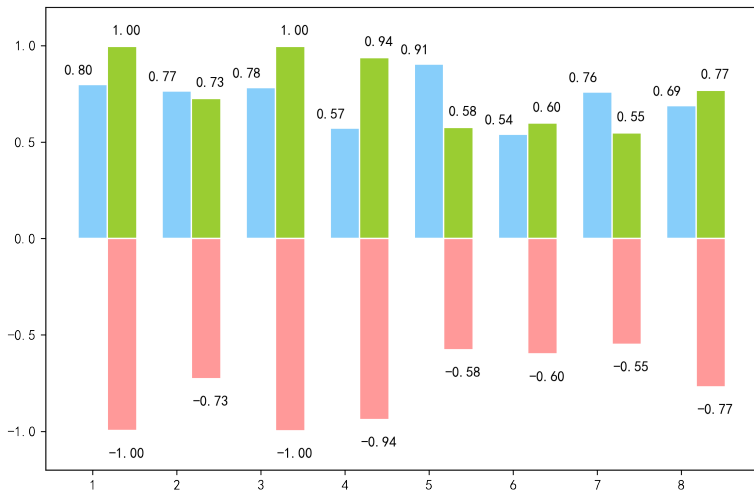
width 是第三个参数，表示柱形图柱子的宽度，一般这是为 1 即可，默认为 0.8，这样让柱子之间有适当空隙，显得美观一点。width 也可以是一个 array 数组；bottom 为柱形图底边的 Y 坐标；color 为柱形图填充的颜色，也可以是一个 array 数组；edgecolor 是柱子边线的颜色，也可以是一个 array 数组；linewidth 为柱形图边线的宽度，也可以是一个 array 数组；xerr 和 yerr 表示 x 轴或者 y 轴柱子上的 errorbar，ecolor 表示这些 errorbar 的颜色。capsize 决定这些 errorbar 的长度；align 设置 plt.xticks() 函数中的标签的位置，有两个可选项 { 'edge' , 'center' }；orientation 表示这些柱子的方向，也有两个选项 { 'vertical' , 'horizontal' }；log 设定布尔值，默认是 False，如果是 True 的话，把坐标轴取对数。

我们还可以使用其他一些辅助命令。比如，使用 `plt.title` 为图形可以添加标题；使用 `plt.xticks` 设置横轴的标签值；使用 `plt.legend` 添加图例。注意，这个命令的参数必须为元组，比如：`legend((line1, line2, line3), ( 'label1' , 'label2' , 'label3' ))`。我们也可以使用 `plt.xlim(a,b)` 函数设置横轴的范围，使用 `plt.ylim(a,b)` 函数设置纵轴的范围。下面我们看一下具体的例子。

垂直柱状图示例

```
plt.figure(1, figsize=(9, 6))
n = 8
X = np.arange(n)+1
Y1 = np.random.uniform(0.5, 1.0, n)
Y2 = np.random.uniform(0.5, 1.0, n)
plt.bar(X, Y1, width = 0.35, facecolor = 'lightskyblue', edgecolor = 'white')
plt.bar(X+0.35, Y2, width = 0.35, facecolor = 'yellowgreen', edgecolor = 'white')
plt.bar(X+0.35, -Y2, width = 0.35, facecolor='#ff9999', edgecolor='white')
plt.figure(1)
for x,y in zip(X,Y1):
    plt.text(x, y+0.05, '%.2f' % y, ha='center', va= 'bottom')
for x,y in zip(X,Y2):
    plt.text(x+0.4, y+0.05, '%.2f' % y, ha='center', va= 'bottom')
for x,y in zip(X,Y2):
    plt.text(x+0.4, -y-0.15, '%.2f' % -y, ha='center', va= 'bottom')
plt.ylim(-1.2,1.2)
plt.show()
```


Figure: 垂直柱状图示例



注意：水平柱状图 `plt.barh`，属性中宽度 `width` 变成了高度 `height`。另外，`facecolor` 表示柱状图里填充的颜色，`edgecolor` 表示边框的颜色。如果想把一组数据打到下边，只需要在数据前使用负号即可。比如，`plt.bar(X, -Y2, width=width, facecolor='#ff9999', edgecolor='white')`。

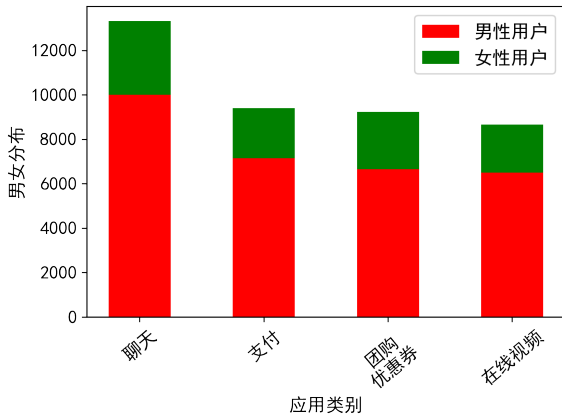
下面的例子可以看看 bottom 的用法。

```
plt.rc('font', family='SimHei', size=13)

num = np.array([13325, 9403, 9227, 8651])
ratio = np.array([0.75, 0.76, 0.72, 0.75])
men = num * ratio
women = num * (1 - ratio)
x = ['聊天', '支付', '团购\优惠券', '在线视频']

width = 0.5
idx = np.arange(len(x))
plt.bar(idx, men, width, color='red', label='男性用户')
plt.bar(idx, women, width, bottom=men, color='green', label='女性用户')
plt.xlabel('应用类别')
plt.ylabel('男女分布')
plt.xticks(idx+width/2, x, rotation=40)
plt.legend()
```

Figure: bottom 用法示例



下面这个例子可以看出 errorbar 的用法，也可以看到 bottom 的作用。

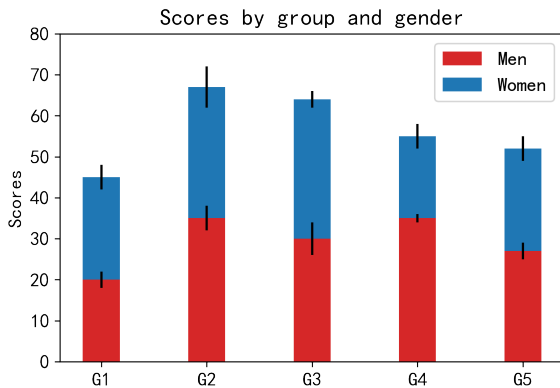
```
N = 5
menMeans = (20, 35, 30, 35, 27)
womenMeans = (25, 32, 34, 20, 25)
menStd = (2, 3, 4, 1, 2)
womenStd = (3, 5, 2, 3, 3)
ind = np.arange(N) # the x locations for the groups
width = 0.35 # the width of the bars: can also be len(x) sequence

p1 = plt.bar(ind, menMeans, width, color='#d62728', yerr=menStd)
p2 = plt.bar(ind, womenMeans, width, bottom=menMeans, yerr=womenStd)

plt.ylabel('Scores')
plt.title('Scores by group and gender')
plt.xticks(ind, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.yticks(np.arange(0, 81, 10))
plt.legend((p1[0], p2[0]), ('Men', 'Women'))

plt.show()
```

Figure: Errorbar 用法示例



饼图是将各项的大小与总和的比例显示在一张“饼”中，以“饼”的大小来确定每一项的占比。饼图可以比较清楚地反映出部分与部分、部分与整体之间的比例关系，易于显示每组数据相对于总数的大小，显示方式直观。绘制饼图的函数为 `pie`，其常用的参数及说明如下表所示。

Table: 饼图常用参数

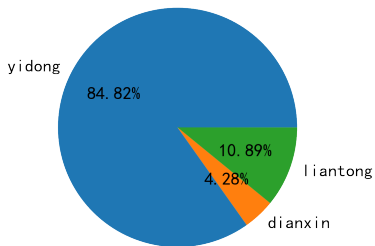
参数名称	说明
<code>x</code>	数组 (array)。表示用于绘制饼图的数据。
<code>explode</code>	数组 (array)。表示指定项距离饼图圆心为 <code>n</code> 个半径。
<code>labels</code>	数组 (array)。指定每一项的名称。
<code>color</code>	特定字符串 (string) 或包含颜色字符串的数组 (array)。表示饼图颜色。
<code>autopct</code>	特定字符串 (string)。指定数值的显示方式。
<code>pctdistance</code>	浮点数 (float)。指定每一项的比例 <code>autopct</code> 和距离圆心的半径。
<code>labeldistance</code>	浮点数 (float)。指定每一项的名称 <code>labels</code> 和距离圆心的半径。
<code>radius</code>	浮点数 (float)。表示饼图的半径。

下面再来看一个绘制饼图的例子。

```
df_phone_prop = [0.8482, 0.0428, 0.1089]
fig = plt.figure()
plt.pie(df_phone_prop, labels=['yidong', 'dianxin', 'liantong'], autopct='%1.2f%%')
plt.title("phone operator")
plt.show()
```


Figure: 饼图示例

phone operator



箱线图也称盒子图，用来刻画数据的分布情况，最早是有 John W.Tukey 在 1969 年提出的。箱线图有两个组成部分，一个是“箱”，一个是“线”。

在 pyplot 中，绘制箱线图的函数为 boxplot, 其常用参数及说明如下所示。

Table: 箱线图常用参数

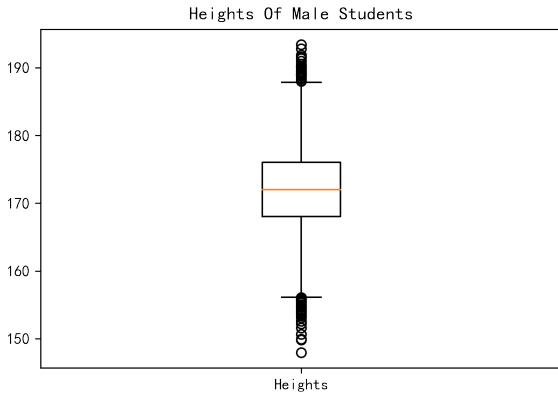
参数名称	说明
x	数组 (array)。表示用于绘制箱线图的数据。
notch	布尔值 (boolean)。表示中间箱体是否有缺口。
sym	特定字符串 (string)。指定异常点形状。
vert	布尔值 (boolean)。表示图形是纵向或者横向。
positions	数组 (array)。表示图形位置。
widths	数值或者数组 (array)。表示每个箱体的宽度。
labels	数组 (array)。指定每一个箱线图的标签。
meanline	布尔值 (boolean)。表示是否显示均值线。

我们来看下面的例子。先来产生一组容量为 10000 的学生身高。

```
from numpy.random import normal
heights = [ ]
N = 10000
for i in range(N):
    while True:
        height = normal(172, 6)
        if 0 < height: break
    heights.append(height)
```

```
plt.boxplot([heights], labels=['Heights'])
plt.title('Heights Of Male Students')
plt.show()
```

Figure: 箱线图示例



概率图用来判断随机样本是否来自某一特定总体（默认是正态总体）。

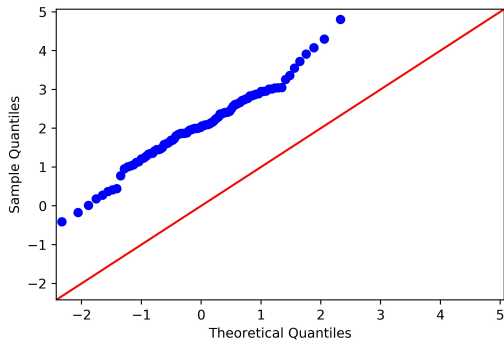
我们看看用 Python 做出概率图的例子。

```
from statsmodels.graphics.gofplots import qqplot

#样本1, 来自正态分布, 均值为2, 标准差为1
data1 = np.random.normal(loc = 2, scale = 1.0, size = 100)

#默认和标准正态分布比较
qqplot(data1, line = '45', dist = 'norm')
plt.show()
```

Figure: 概率图示例



```
#样本2, 来自正态分布, 均值为0, 标准差为1
```

```
data2 = np.random.normal(loc = 0, scale = 1.0, size = 100)
```

```
qqplot(data2, line = '45', dist = 'norm')
```

```
plt.show()
```

Figure: 概率图示例 2

